Számítógépes Hálózatok

8-9-10. (mininet) gyakorlat

MININET

Alapok, router interface-ek (Bead 1. alfeladat)

Mininet setup (virtualbox)

1. Töltsük le a virtuális gépet (2Gb):
https://ikelte-my.sharepoint.com/:u:/g/personal/afbdpk_inf_elte_hu/EfAwBnSEDudLuU6cR-sb4WMBu1atz4ll3Ivsmg1gQDtb8Q?download=1
2. Csomagoljük ki a zip-et (6.6Gb)
3. Oracle VirtualBox-ban készítsünk egy új virtuális gépet
   ISO nem kell
   Típusa: Linux - Ubuntu(64bit) -> Next
   Hardware részen adjunk neki valamennyit
   Virtual Disk: Meglévő használata opció
   -> a kicsomagolt vhd adjuk meg
4. Settings -> System -> Processor -> Enable PAE/NX
5. Start Ubuntu VM

Mininet adaptív képernyő

Ha a virtuális Ubuntu képernyője nem tölti ki a teljes VirtualBox ablakot.

1. Miközben fut az Ubuntu VM:
   Devices -> Insert Guest additions CD image
   -> jelszó (*"networks"*), meg kell várni míg települ és végén enter
2. Shutdown Ubuntu VM
3. (Opcionális, géptől függ) Oracle:
   Settings -> Display -> Enable 3D acceleration
   (ha utána csak a háttérkép látható és más nem, akkor kapcsold ki)
4. Miközben fut az Ubuntu VM:
   File -> Preferences -> Display -> Maximum Guest Screen Size:
   Hint -> a géped felbontása (pl.1920x1080) -> OK
5. Start Ubuntu VM -> tedd ki teljes képenyőre
6. Miközben fut az Ubuntu VM:
   View -> Auto-resize Guest Display

Mininet ismerkedés 1

- Egy Ubuntu 14 virtuális gépünk van. Felhasználó/jelszó: ***networks***
- Váltsunk rootra:

```
networks@networks:~$ sudo su
```

- Listázzuk az alábbi könyvtárat:

```
root@networks:/home/networks# cd ComputerNetworks/L2-switching
```

- Listázzuk az alábbi könyvtárat:

```
root@networks:/home/networks# ls
```

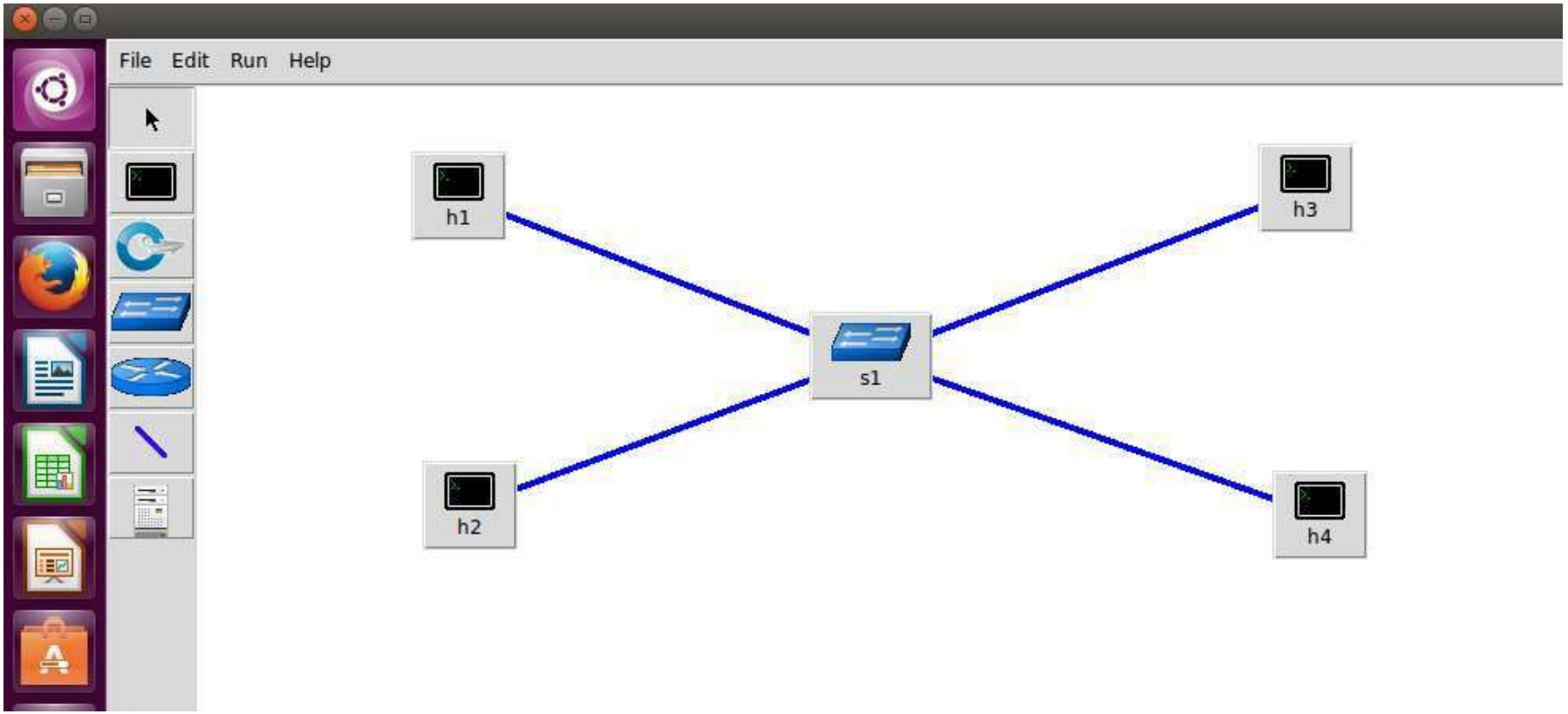
Mininet ismerkedés 2

- A test 1 topológiához két fájl tartozik:
- test1.mn: Leírja a topológiát
- test1.py: futtatható script
- Indítsuk el a minieditet:

```
root@networks:/home/networks/mininet/examples#
                                        python miniedit.py
```

Mininet ismerkedés 3

- A file menü open pontjából megtudjuk nyitni a .mn kiterjesztésű fájlokat. Nyissuk is meg a test1-et.



Mininet ismerkedés 4

- A file menü Export Level2 Script pontjával létrehozhatjuk a hálózathoz tartozó python scriptet. Ezt most nem kell megtennünk, hiszen ott a .py fájl.
- Vizsgáljuk meg a test1 scriptet (vim test1.py)

```
from mininet.net import Mininet
from mininet.node import Controller, RemoteController, OVSController
from mininet.node import CPULimitedHost, Host, Node
from mininet.nodelib import LinuxBridge
from mininet.node import IVSSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel, info
from mininet.link import TCLink, Intf
from subprocess import call

def myNetwork():

    net = Mininet( topo=None,
                   build=False,
                   ipBase='10.0.0.0/8')

    info( '*** Adding controller\n' )
    info( '*** Add switches\n')
    s1 = net.addSwitch('s1', cls=LinuxBridge, failMode='standalone')

    info( '*** Add hosts\n')
    h3 = net.addHost('h3', cls=Host, ip='10.0.0.3', defaultRoute=None)
    h2 = net.addHost('h2', cls=Host, ip='10.0.0.2', defaultRoute=None)
    h4 = net.addHost('h4', cls=Host, ip='10.0.0.4', defaultRoute=None)
    h1 = net.addHost('h1', cls=Host, ip='10.0.0.1', defaultRoute=None)

    info( '*** Add links\n')
    net.addLink(h2, s1)
    net.addLink(s1, h3)
    net.addLink(s1, h4)
    h1s1 = {'bw':10}
    net.addLink(h1, s1, cls=TCLink , **h1s1)

    info( '*** Starting network\n')
    net.build()
    info( '*** Starting controllers\n')
    for controller in net.controllers:
        controller.start()

    info( '*** Starting switches\n')
    net.get('s1').start([])

    info( '*** Post configure switches and hosts\n')

    CLI(net)
    net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    myNetwork()
```

Python script

- Egy LinuxBridge-t definiálunk, amellyel futtatni tudjuk a feszítőfa algoritmust (SpanningTreeProtocol, STP) hurkok kezelésére
- Hozzáadunk hosztokat is, privát IP címekkel
- Végül összekötjük ezeket a topológia alapján
- A h1-s1 kapcsolat sávszélessége: 10 Mbps(alapból elvileg nem limitált, a TCLinkosztály azért kell, hogy limitálni tudjuk)
- Futtatás:
-

```
root@networks:/home/networks/ComputerNetworks/L2-switching#
python test1.py
mininet>
```

Linkek tulajdonságai

- Hálózati linkek tulajdonságai:
- késleltetés (delay), csomagvesztés (loss), sávszélesség (bw)
- A mininetes pythonscriptben:
- linkopts= {'bw':10, 'delay':'5ms’, loss=10 }
- # bw: sávszélesség Mbps-ben
- # delay: késleltetés mértékegységgel: ms, s, us, stb.
- # loss: 0-100 közötti egész, csomagvesztés százalék
-

```
net.addLink(h1, r1, cls=TCLink, **linkopts)
```

Elérhető eszközök - Hostok

- Elérhető csomópontok:

```
mininet> nodes
```

- A h1 h2 hostokon elindíthatunk egy-egy terminált:

```
mininet> xterm h1 h2
```

- Itt lekérhetők az interface adatok:

```
# ifconfig
```

- Pingessük meg h1-ről h2-őt:

```
# ping10.0.0.2
```

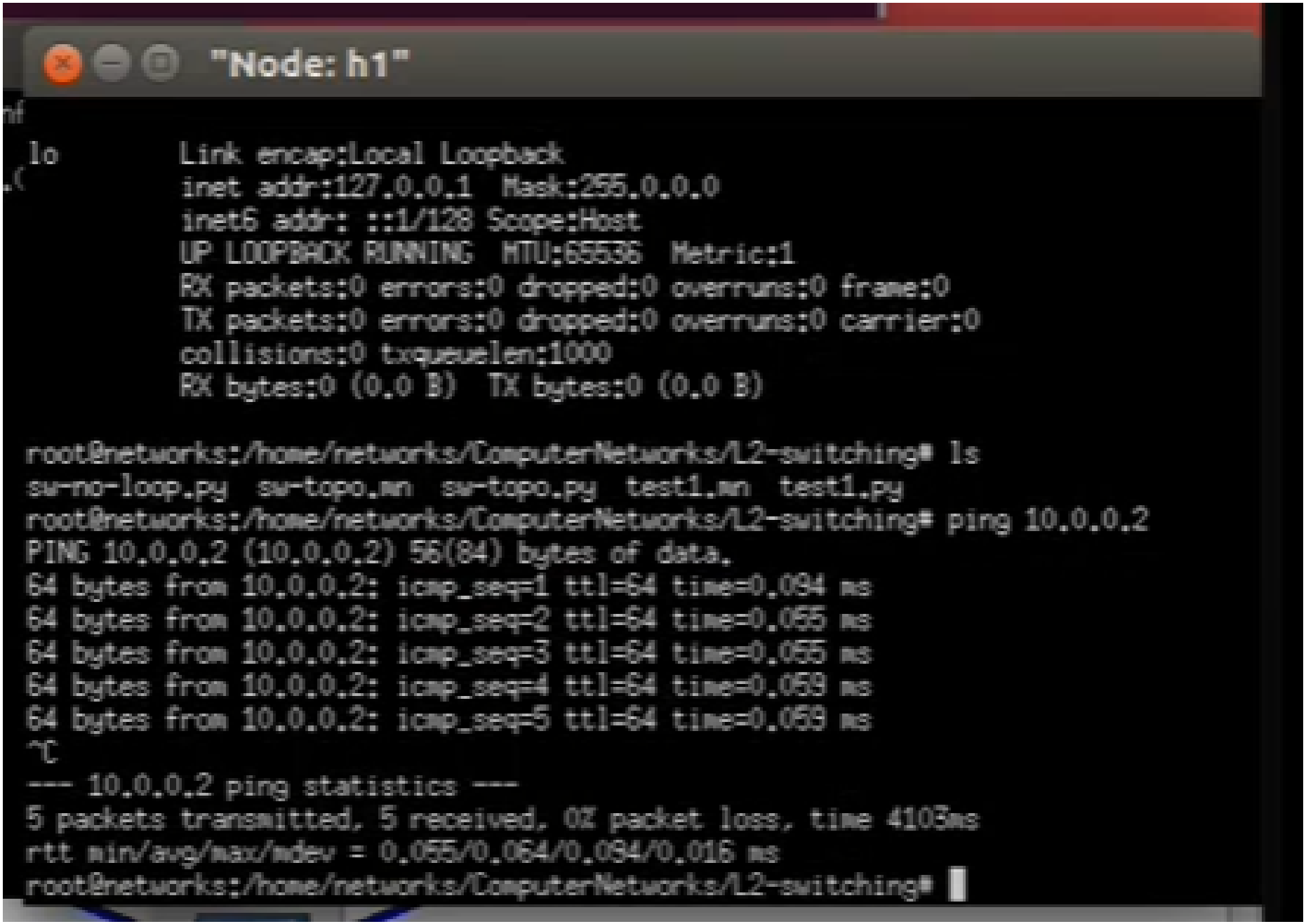
```
root@networks:/home/networks/ComputerNetworks/L2-switching# python test1.py
*** Adding controller
*** Add switches
*** Add hosts
*** Add links
(10.00Mbit) (10.00Mbit) *** Starting network
*** Configuring hosts
h3 h2 h4 h1
*** Starting controllers
*** Starting switches
*** Post configure switches and hosts
*** Starting CLI:
mininet> nodes
available nodes are:
h1 h2 h3 h4 s1
mininet> xterm h1 h2
mininet>
```

"Node: h2"

```
root@networks:/home/networks/ComputerNetworks/L2-switching# ifconf
h2-eth0   Link encap:Ethernet  HWaddr ce:8d:60:ce:96:92
          inet addr:10.0.0.2  Bcast:10.255.255.255  Mask:255.0.0.0
          inet6 addr: fe80::cc8d:60ff:fece:9692/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:79 errors:0 dropped:0 overruns:0 frame:0
          TX packets:10 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:9628 (9.6 KB)  TX bytes:796 (796.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

root@networks:/home/networks/ComputerNetworks/L2-switching#
```

"Node: h1"

```
root@networks:/home/networks/ComputerNetworks/L2-switching# ifconfig
h1-eth0   Link encap:Ethernet  HWaddr 5a:e5:20:ad:84:af
          inet addr:10.0.0.1  Bcast:10.255.255.255  Mask:255.0.0.0
          inet6 addr: fe80::58e5:20ff:fead:84af/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:79 errors:0 dropped:0 overruns:0 frame:0
          TX packets:10 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:9628 (9.6 KB)  TX bytes:796 (796.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

root@networks:/home/networks/ComputerNetworks/L2-switching#
```



- Mivel a hostok létrehozott klónok, ezért minden más parancs is m ködik. pl.: ***ls***

Hasznos mininet parancsok

- Közvetlenül a mininetből is pingethetünk:

```
mininet> h1 ping h2
```

- Egyszer topológia vizsgálat:

```
mininet> pingall
```

- ilépés:

```
mininet> exit
```

```
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.050 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.067 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.068 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.052 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.049 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.050 ms
^C
--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5101ms
rtt min/avg/max/mdev = 0.049/0.056/0.068/0.008 ms
mininet> pingall
*** Ping: testing ping reachability
h3 -> h2 h4 h1
h2 -> h3 h4 h1
h4 -> h3 h2 h1
h1 -> h3 h2 h4
*** Results: 0% dropped (12/12 received)
```

Elérhető eszközök - Switchek

- A switchek beállításai:

```
mininet> sh brctl show
```

  - *Sh: shell parancs*
  - *brctl jelentése: bridge control*
- Látszik, hogy nincs engedélyezve az STP
  - *STP jelentése: Spanning Tree Protocol (feszítőfa)*

```
mininet> nodes
available nodes are:
h1 h2 h3 h4 s1
mininet> sh brctl show
bridge name     bridge id               STP enabled     interfaces
s1              8000.124ca2c211d7       no              s1-eth1
                                                        s1-eth2
                                                        s1-eth3
                                                        s1-eth4
mininet>
```

Hálózati forgalom generálás

- Az s1 switch forwarding tábláját lekérdezhetjük a mininetkonzolban:

```
mininet> sh brctl showmacs s1
```

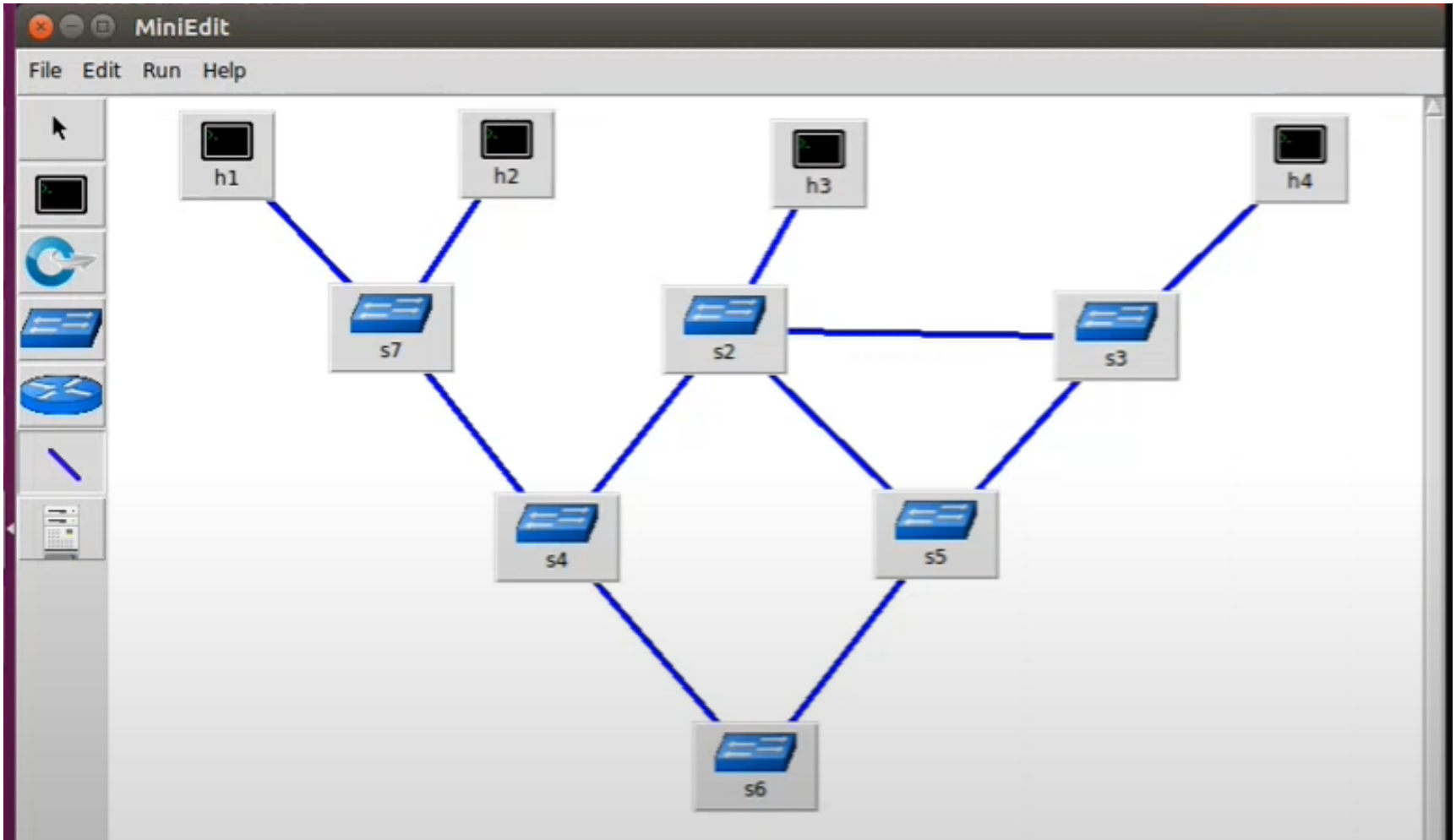
- Figyeljük a forgalmat minden interfészen:

```
mininet> sh tcpdump –n –i any
```

```
mininet> sh brctl showmacs s1
port no mac addr                is local?       ageing timer
  4     3a:50:51:dc:53:d4       yes                0.00
  4     3a:50:51:dc:53:d4       yes                0.00
  4     5a:e5:20:ad:84:af       no                64.69
  3     86:0d:cd:1b:9a:44       yes                0.00
  3     86:0d:cd:1b:9a:44       yes                0.00
  1     8e:26:ac:73:41:f9       yes                0.00
  1     8e:26:ac:73:41:f9       yes                0.00
  1     ce:8d:60:ce:96:92       no                81.08
  3     ce:a3:51:16:26:f0       no                97.46
  2     f2:d9:12:ad:b7:db       no               105.65
  2     f6:07:25:8f:86:1d       yes                0.00
  2     f6:07:25:8f:86:1d       yes                0.00
mininet> s1 tcpdump -n -i any
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on any, link-type LINUX_SLL (Linux cooked), capture size 262144 bytes
14:25:16.870362 IP6 fe80::8c26:acff:fe73:41f9 > ff02::2: ICMP6, router solicitat
ion, length 16
```

Feladat 2: Sw-topo

- Nyissuk meg a minieditben az sw-topo.mn fájlt
- Mit veszünk észre?
  - *A topológia **kört** tartalmaz, ezért a felderítés kapcsán a csomagok beakadnak és "csomagvihar" keletkezik.*



Használhatatlan hálózat

- h1 és h össze vannak kötve, akkor működnie kell a pingnek igaz?

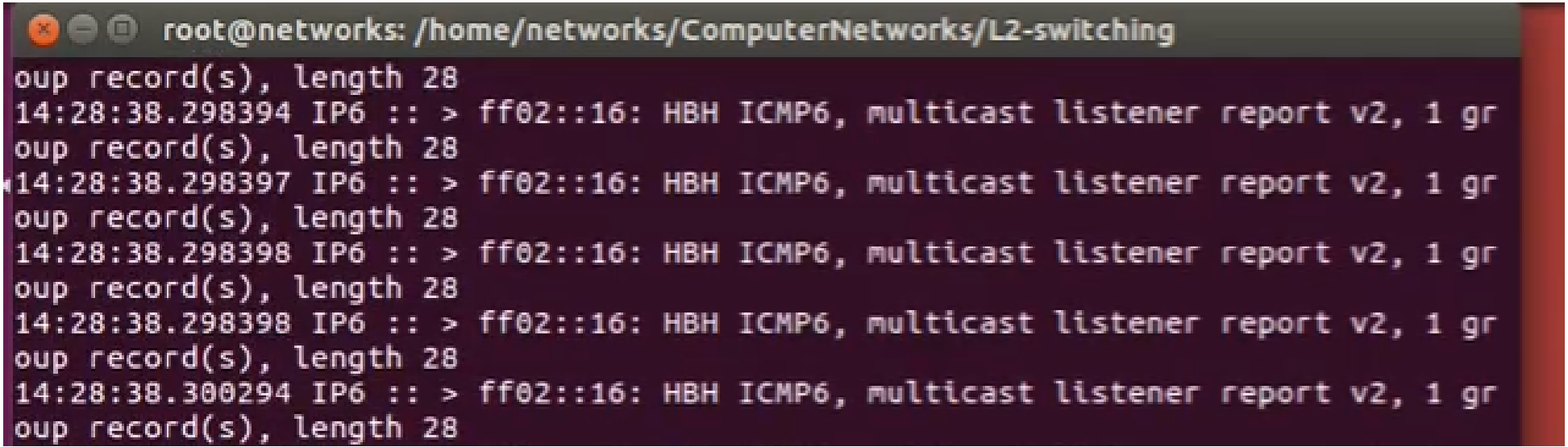
```
mininet> h1 ping h4
```

```
mininet> h1 ping h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable
```

- Nem, sok csomag elveszik és óriási a késés
- Figyeljük a forgalmat minden interfészen:

```
mininet> sh tcpdump –n –i any
```

- Tcpdump-pal megfigyelhetjük, hogy broadcast üzenetek próbálják folyamatosan felderíteni a hálózatot:

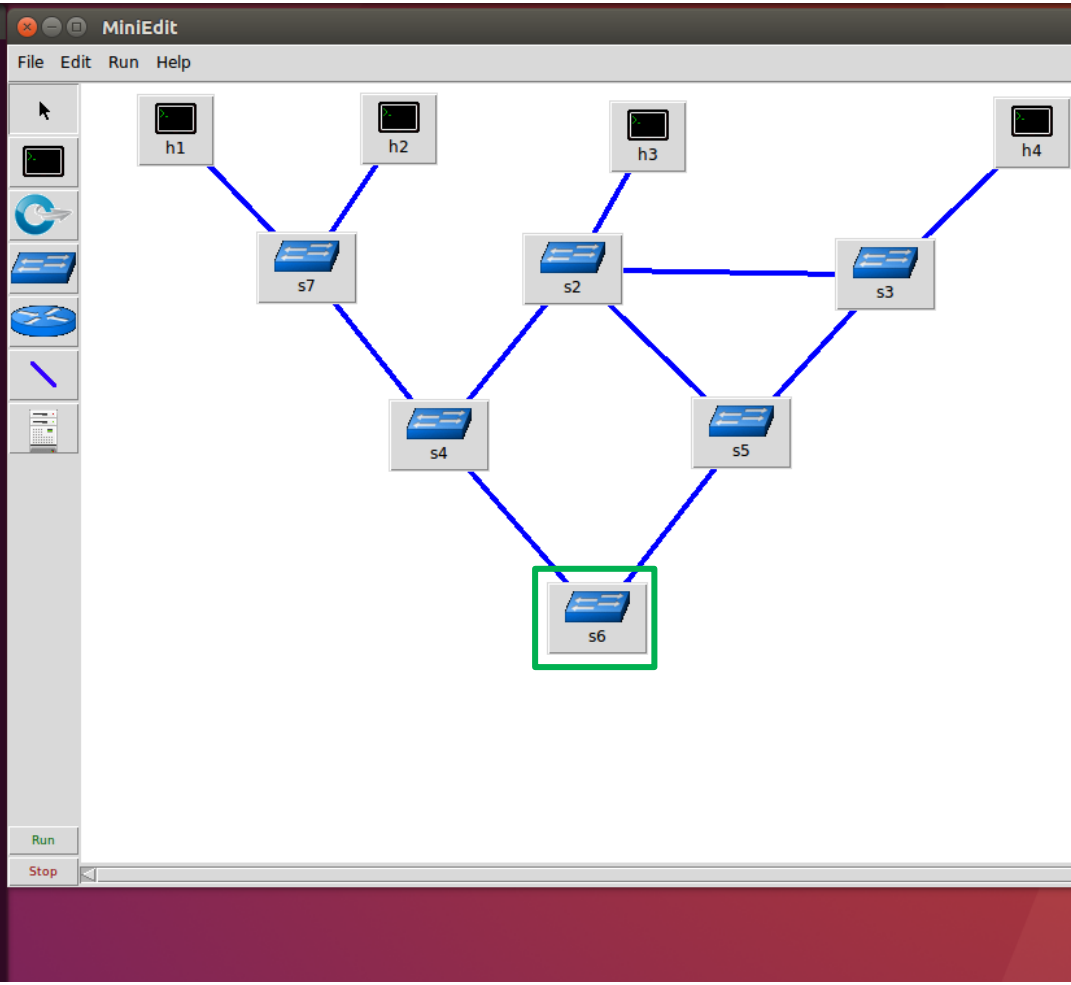


Hálózat kijavítása - STP

- Hurkokat tartalmaz. Valahogy fel kell oldanunk a logaikai hurkokat
- Használjuk egy algoritmust, amely a hálózatból feszítőfát hoz létre, azért hogy ne legyen benne kör.

```
mininet> sh brctl show
```

```
root@networks: /home/networks/ComputerNetworks/L2-switching
*** Starting CLI:
mininet> sh brctl show
bridge name     bridge id               STP enabled     interfaces
s2              8000.32c7c790adac       yes             s2-eth1
                                                        s2-eth2
                                                        s2-eth3
                                                        s2-eth4
s3              8000.369e11b8a7b3       yes             s3-eth1
                                                        s3-eth2
                                                        s3-eth3
s4              8000.4a9490f7e79c       yes             s4-eth1
                                                        s4-eth2
                                                        s4-eth3
s5              8000.2e073f193228       yes             s5-eth1
                                                        s5-eth2
                                                        s5-eth3
s6              8000.1ea24d709a2f       yes             s6-eth1
                                                        s6-eth2
s7              8000.2a410c04c349       yes             s7-eth1
                                                        s7-eth2
                                                        s7-eth3

mininet> sh brctl showstp s2
s2
 bridge id              8000.32c7c790adac
 designated root        8000.1ea24d709a2f
 root port                 2                    path cost                  4
 max age                  20.00                 bridge max age            20.00
 hello time                2.00                 bridge hello time          2.00
 forward delay            15.00                 bridge forward delay      15.00
 ageing time             300.00
 hello timer               0.00                 tcn timer                  0.00
 topology change timer     0.00                 gc timer                 144.38
 flags


s2-eth1 (1)
 port id                8001                    state                forwarding
 designated root        8000.1ea24d709a2f       path cost                     2
 designated bridge      8000.32c7c790adac       message age timer          0.00
 designated port        8001                    forward delay timer        0.00
 designated cost           4                    hold timer                 0.38
 flags
```



ijavított hálózat

- Indítsuk újra az stp kapcsolóval:

```
# python sw-topo.py --stp
```

- Switch állapot:

```
mininet> sh brctl show
```

- STP információ az s2 switchhez:

```
mininet> sh brctl showstp s2
```

- i a designated root, designated bridge
  Mely portok blokkoltak
  - *Designated **root**: a fa gyökere, designated **bridge**: a roothoz "vezető" switch*
  - *Nézzük meg a **state** állapotot, ha forwarding akkor működik*

```
mininet> sh brctl showstp s2
s2
 bridge id              8000.1a102cd6b3fd
 designated root        8000.1a102cd6b3fd
 root port                 0                    path cost                  0
 max age                  20.00                 bridge max age            20.00
 hello time                2.00                 bridge hello time          2.00
 forward delay            15.00                 bridge forward delay      15.00
 ageing time             300.00
 hello timer               0.23                 tcn timer                  0.00
 topology change timer     0.00                 gc timer                 198.23
 flags


s2-eth1 (1)
 port id                8001                    state                forwarding
 designated root        8000.1a102cd6b3fd       path cost                     2
 designated bridge      8000.1a102cd6b3fd       message age timer             0.00
 designated port        8001                    forward delay timer           0.00
 designated cost           0                    hold timer                    0.00
 flags

s2-eth2 (2)
 port id                8002                    state                forwarding
 designated root        8000.1a102cd6b3fd       path cost                     2
 designated bridge      8000.1a102cd6b3fd       message age timer             0.00
 designated port        8002                    forward delay timer           0.00
 designated cost           0                    hold timer                    0.00
 flags

s2-eth3 (3)
 port id                8003                    state                forwarding
 designated root        8000.1a102cd6b3fd       path cost                     2
 designated bridge      8000.1a102cd6b3fd       message age timer             0.00
 designated port        8003                    forward delay timer           0.00
 designated cost           0                    hold timer                    0.00
 flags

s2-eth4 (4)
 port id                8004                    state                forwarding
 designated root        8000.1a102cd6b3fd       path cost                     2
 designated bridge      8000.1a102cd6b3fd       message age timer             0.00
 designated port        8004                    forward delay timer           0.00
 designated cost           0                    hold timer                    0.00
 flags
```

IP címek interfészhez rendelése

- Interfészek lekérdezése:

```
mininet> h1 ifconfig
```

- IP cím interfészhez rendelése:

```
mininet> ip addr add 10.0.2.1/24 dev h1-eth0
```

- A verzió ami régebbről ismerős lehet:

```
mininet> ifconfig h1-eth0 10.0.2.1 netmask 255.255.255.0
```

```
mininet> h1 ifconfig
h1-eth0   Link encap:Ethernet  HWaddr 9a:ae:1e:75:88:5b
          inet addr:10.0.0.1  Bcast:10.255.255.255  Mask:255.0.0.0
          inet6 addr: fe80::98ae:1eff:fe75:885b/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:365 errors:0 dropped:0 overruns:0 frame:0
          TX packets:35 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:37402 (37.4 KB)  TX bytes:2882 (2.8 KB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

Feladat 3: Routes

- A honlapról töltsük le a routes.py és routes.mn fájlokat a VM-en belül.

wornox.web.elte.hu/Telekommunikációs_hálózatok/gyak8-9-10(mininet)/

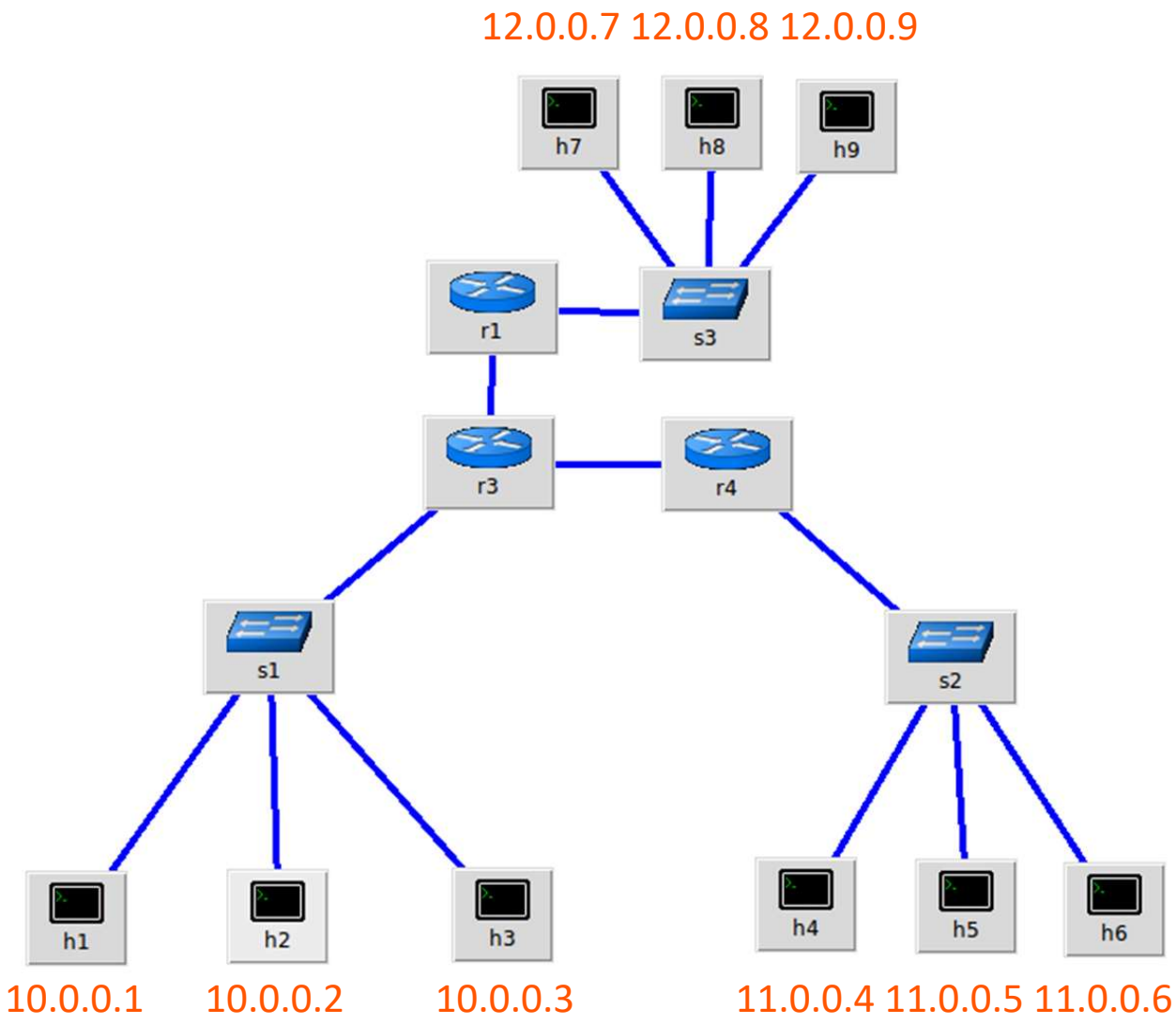
Feladatok,anyagok/VAGY HASZNÁLJUK /gyak8-9-10(mininet)/Mininet fájlok megosztása.txt

- Nyissuk meg a **routes.mn** fájlt *miniedit*-tel, hogy megnézzük a topológiát.
- Nyissuk meg *Geany* vagy *Vim*-el a **routes.py** kódot.

IP címek interfészhez rendelése

Feladat 3.1: Nyerjük ki az alap információkat, a routes.py fájlból!

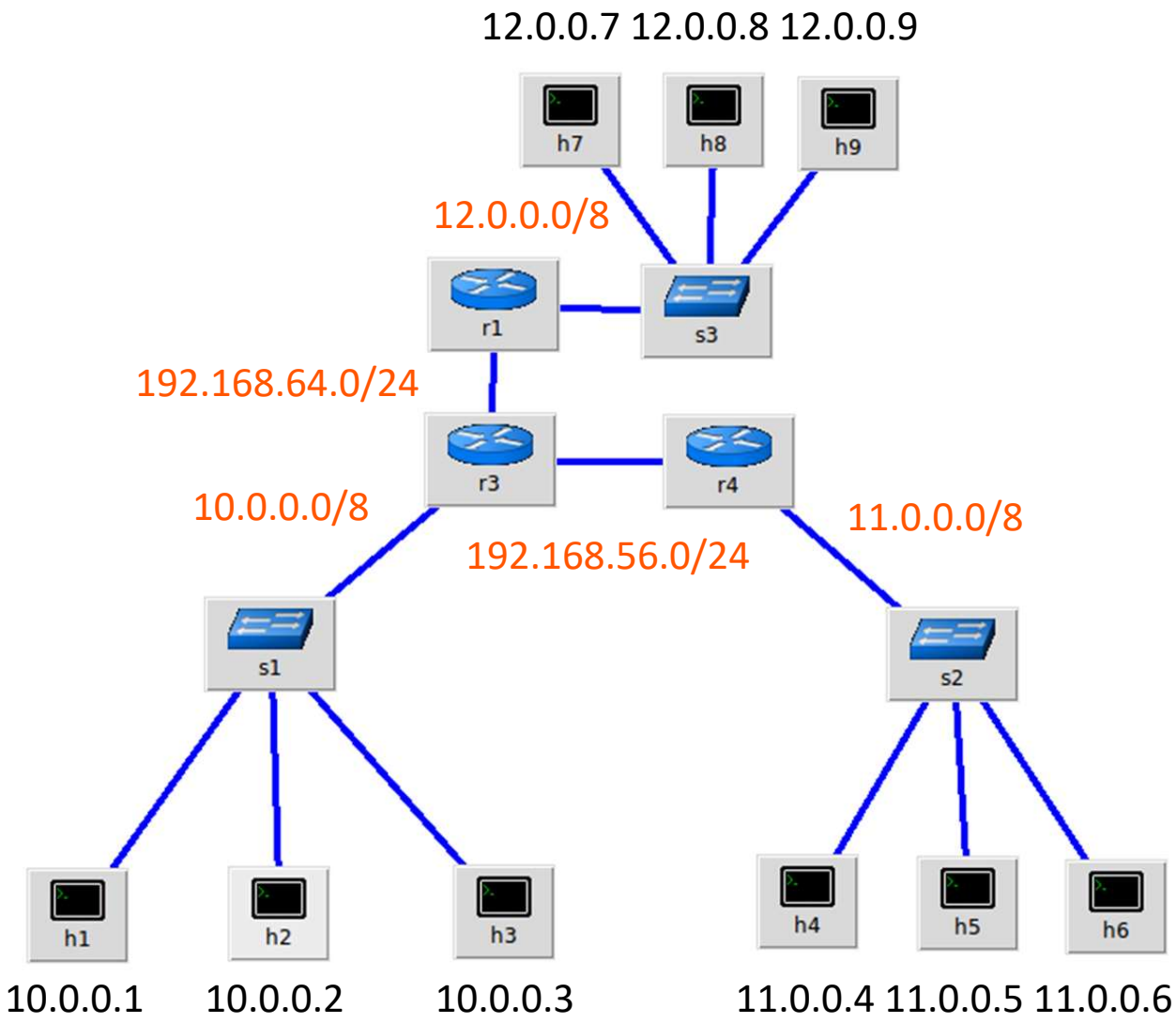
(a narancssárga IP-k a kódból vagy a ifconfigokból kinyert adatok)



IP címek interfészhez rendelése

Feladat 3.2: egészítsük ki a hiányzó részeket!

(a narancssárga IP-k az általunk adott IP-k, figyeljünk az alhálózatokra)



IP címek interfészhez rendelése

Feladat 3.3: Nézzük meg az egyik router interfaceit!

(minineten belül adjuk ki a r3 inconfig parancsot)

```
mininet> r3 ifconfig
lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

r3-eth0   Link encap:Ethernet  HWaddr b6:1d:47:d4:30:5c
          inet6 addr: fe80::b41d:47ff:fed4:305c/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:34 errors:0 dropped:0 overruns:0 frame:0
          TX packets:7 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:3959 (3.9 KB)  TX bytes:586 (586.0 B)

r3-eth1   Link encap:Ethernet  HWaddr 42:d1:d6:c1:5d:2d
          inet6 addr: fe80::40d1:d6ff:fec1:5d2d/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:9 errors:0 dropped:0 overruns:0 frame:0
          TX packets:7 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:766 (766.0 B)  TX bytes:586 (586.0 B)

r3-eth2   Link encap:Ethernet  HWaddr d2:18:c6:7f:11:5b
          inet6 addr: fe80::d018:c6ff:fe7f:115b/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:7 errors:0 dropped:0 overruns:0 frame:0
          TX packets:7 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:586 (586.0 B)  TX bytes:586 (586.0 B)

mininet>
```

IP címek interfészhez rendelése

Feladat 3.3: Nézzük meg a linkek interfaceit!

(minineten belül adjuk ki a links parancsot)

Nézzük például az r3-as routert!

- Keressük meg azokat a sorokat, ahol előfordul az r3.
- Ha megvannak ezek a párok, akkor látható, hogy r3 egy másik eszközzel mely interface-n van összekötve.

10.0.0.0/8 alhálózat : s1 - r3 ->

192.168.56.0/24 alhálózat : r3 - r4 ->

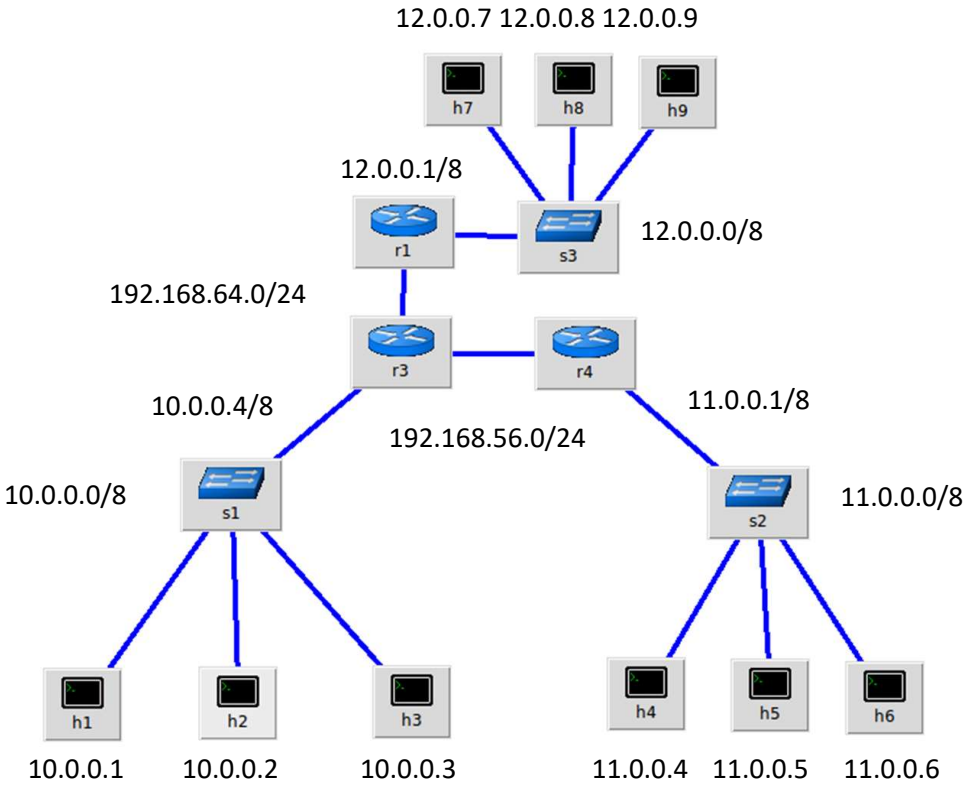
192.168.64.0/24 alhálózat : r1 - r3 ->

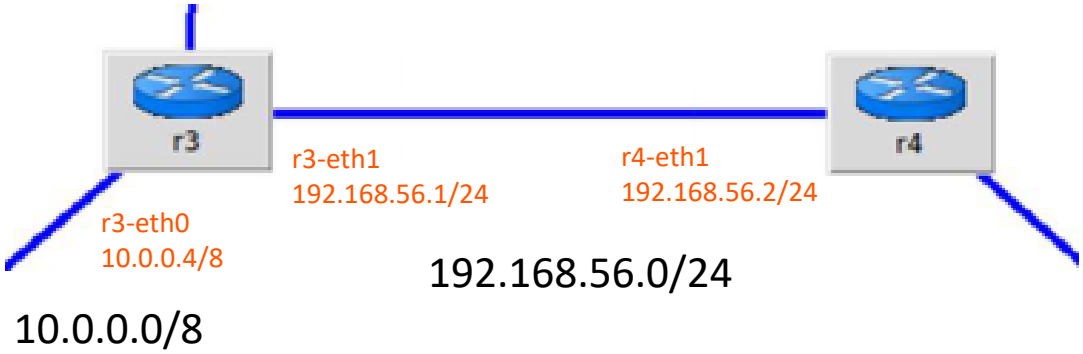
```
*** Starting CLI:
mininet> links
s3-eth1<->h7-eth0 (OK OK)
r1-eth0<->s3-eth2 (OK OK)
s3-eth3<->h8-eth0 (OK OK)
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s1-eth2 (OK OK)
h3-eth0<->s1-eth3 (OK OK)
h4-eth0<->s2-eth1 (OK OK)
h5-eth0<->s2-eth2 (OK OK)
h6-eth0<->s2-eth3 (OK OK)
s1-eth4<->r3-eth0 (OK OK)
s2-eth4<->r4-eth0 (OK OK)
r3-eth1<->r4-eth1 (OK OK)
s3-eth4<->h9-eth0 (OK OK)
r1-eth1<->r3-eth2 (OK OK)
mininet>
```

IP címek interfészhez rendelése

Feladat 3.4: Rendeljük az IP-ket a megfelelelő interfacekhez!

(a narancssárga jelöli az összetartozó példa párokat)





r3 ip addr add 10.0.0.4/8 dev r3-eth0

r3 ip addr add 192.168.56.1/24 dev r3-eth1

r3 ip addr add 192.168.64.1/24 dev r3-eth2

r4 ip addr add 192.168.56.2/24 dev r4-eth1

r4 ip addr add 11.0.0.1/8 dev r4-eth0

r1 ip addr add 192.168.64.2/24 dev r1-eth1

r1 ip addr add 12.0.0.1/8 dev r1-eth0

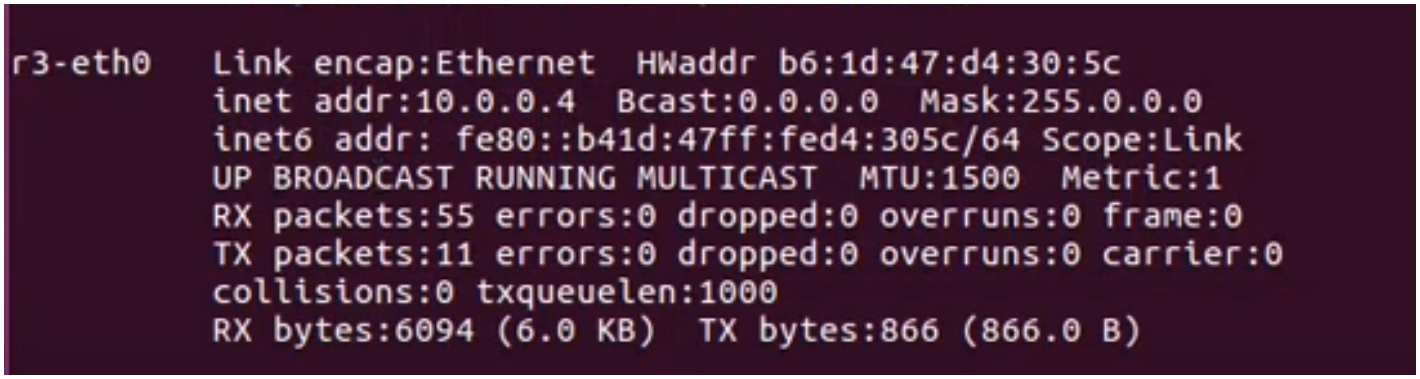
Konfigurálás elkezdése

Nézzük meg az előbbi parancsok hatását

- Adjuk ki az első parancsot:

  r3 ip addr add 10.0.0.4/8 dev r3-eth0
- Nézzük meg hogy mi változott az r3-nál.

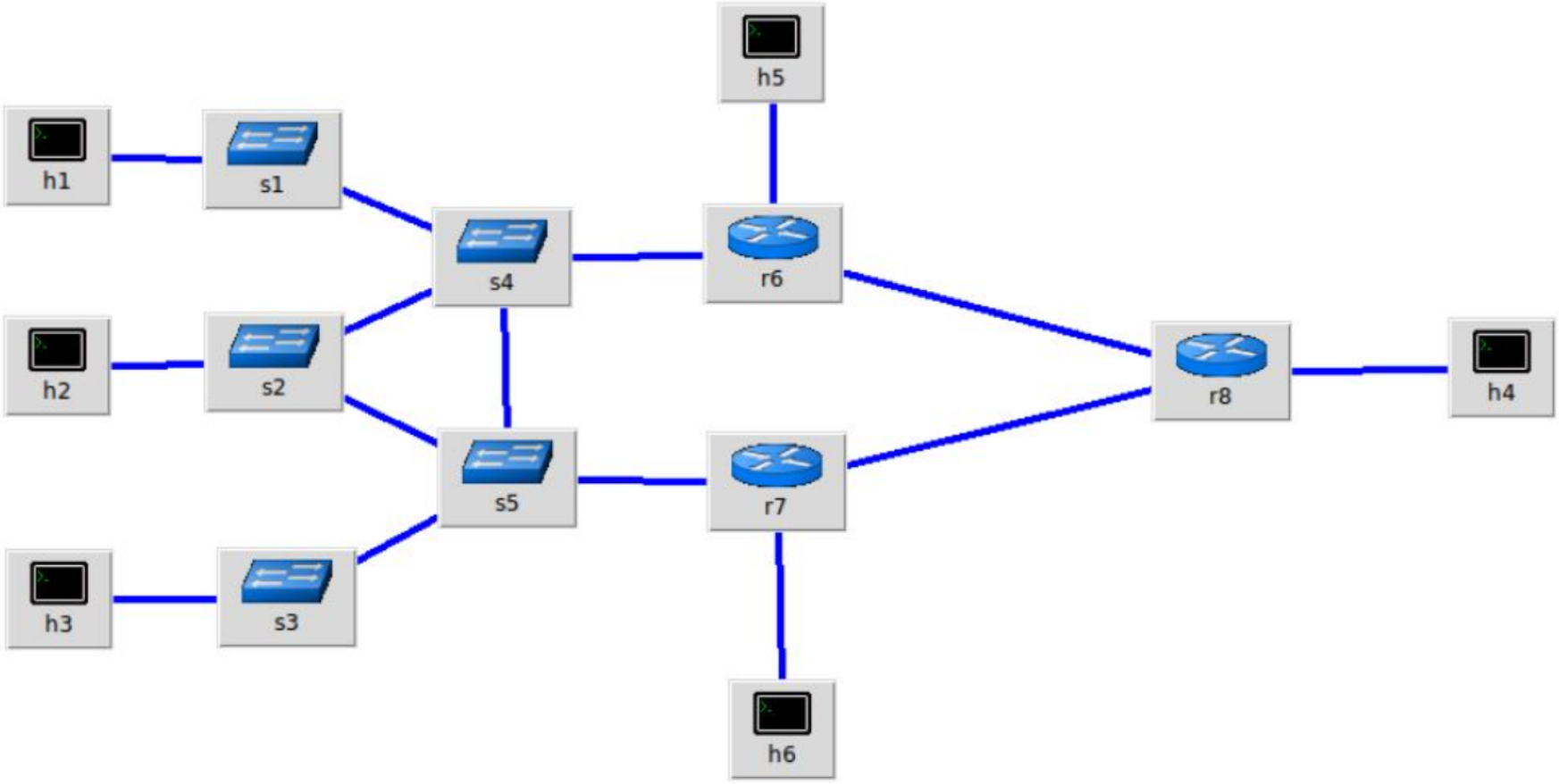
  r3 ifconfig



- r3 router eth0-s interface kapott egy IPv4-es címet.

Beadandó (mininet)

Neptun azonosító: Add a házit!



A homework.py fájl tartalmazza az ábrán látható topológia Mininet-es megvalósítását. A feladatokat ezen a fájlon/topológián kell megoldani, annak módosítása nélkül! A topológia vegyesen tartalmaz hosztokat (hX), switcheket (sY) és routereket (rZ) is.

A beadás formátuma

1db **input.txt** file legyen ami tartalmazza soronkent a kiadott parancsokat. A parancsokat úgy kell megadni, mintha a mininet console-ban adtuk volna ki. pl: 'h1 ping 10.0.0.2' , vagyis a h1 hoszton kiadott 'ping 10.0.0.2' parancs.

Beadandó (mininet)

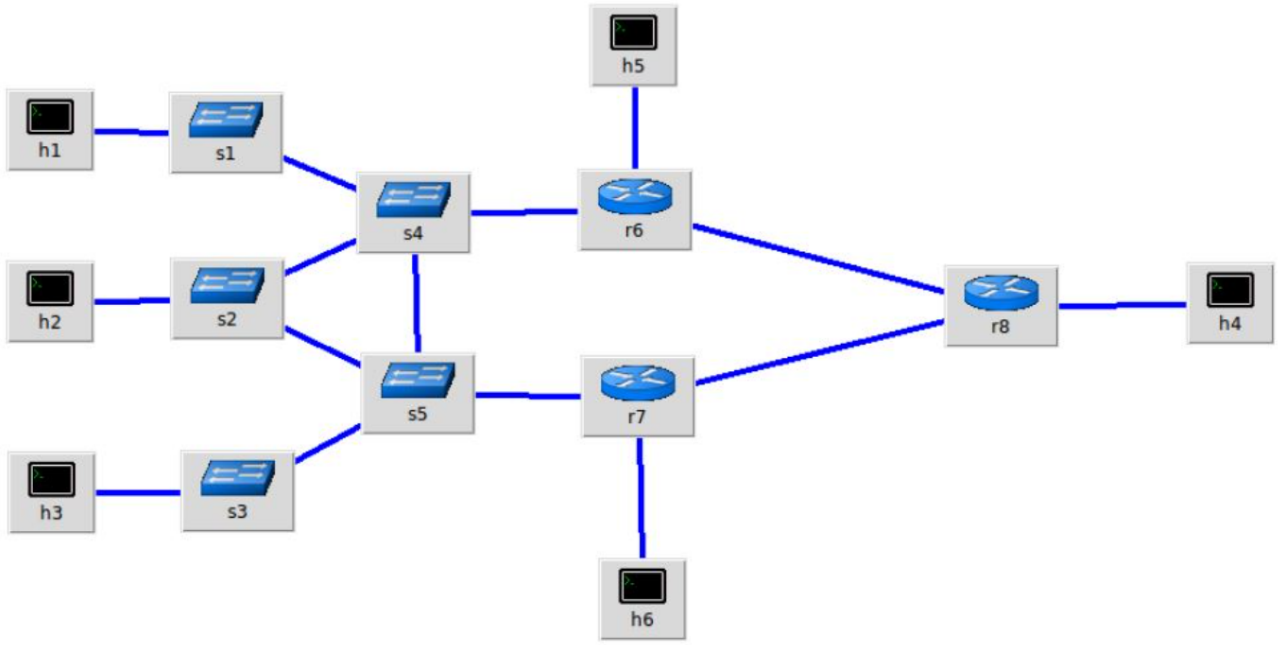
- 1. alfeladat (Router interface-ek)

1. alfeladat

Konfigurálja fel az r6-r8 routerek interfészeit a következő módon:

- r6-s4 irányába: 15.0.0.254/8 legyen. r6-h5 irányába: 25.0.0.254/8 legyen. r6-r8 irányába: 192.168.97.1/24 legyen.
- r7-s5 irányába: 15.0.0.253/8 legyen. r7-h6 irányába: 35.0.0.254/8 legyen. r7-r8 irányába: 192.168.102.1/24 legyen.
- r8-r6 irányába: 192.168.97.2/24 legyen. r8-r7 irányába: 192.168.102.2/24 legyen. r8-h4 irányába: 45.0.0.254/8 legyen.

Neptun azonosító: Add a házit!



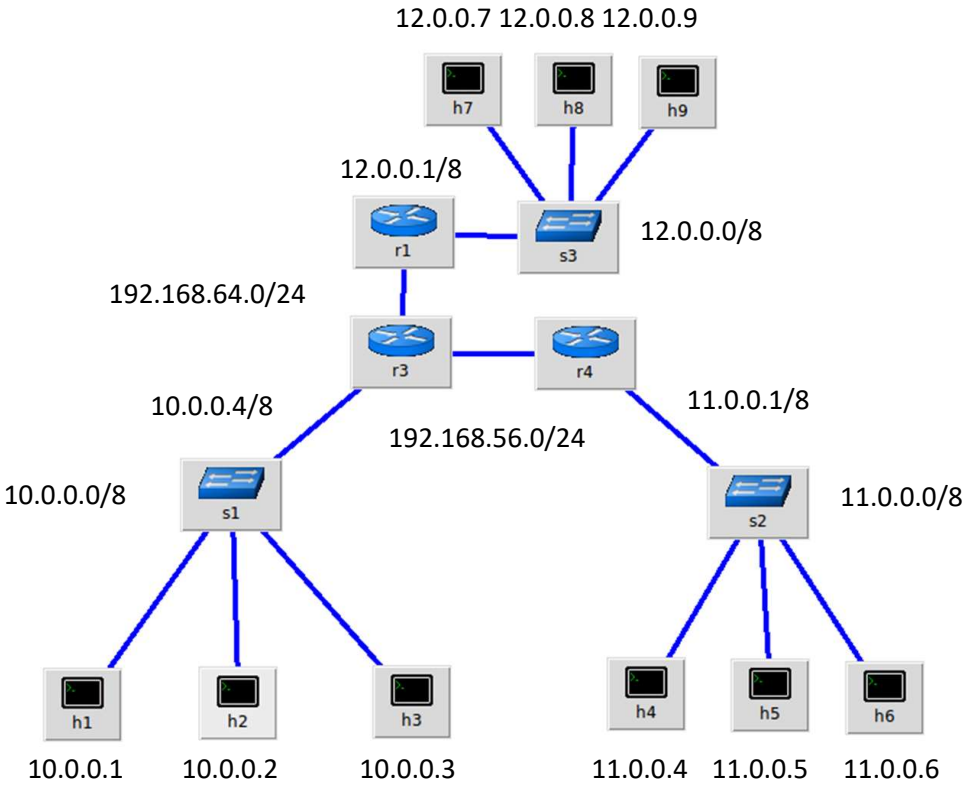
A homework.py fájl tartalmazza az ábrán látható topológia Mininet-es megvalósítását. A feladatokat ezen a fájlon/topológián kell megoldani, annak módosítása nélkül! A topológia vegyesen tartalmaz hosztokat (hX), switcheket (sY) és routereket (rZ) is.

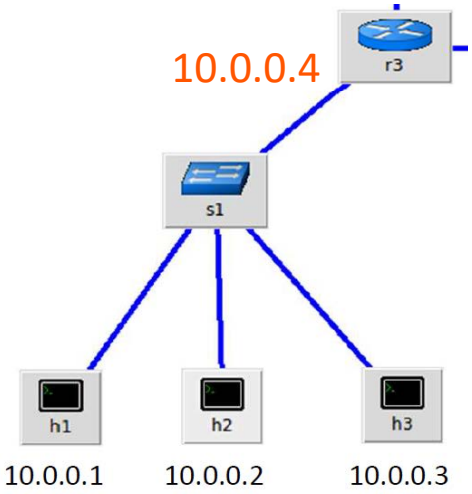
MININET

Default hostok, routing táblák (Bead 2. alfeladat)

Hosztok default routing

Feladat 3.5: Mondjuk meg a hosztoknak, hogy merre kell küldeni a külsős csomagot!





h1 ip route add default via 10.0.0.4

h2 ip route add default via 10.0.0.4

h3 ip route add default via 10.0.0.4

h4 ip route add default via 11.0.0.1

h5 ip route add default via 11.0.0.1

h6 ip route add default via 11.0.0.1

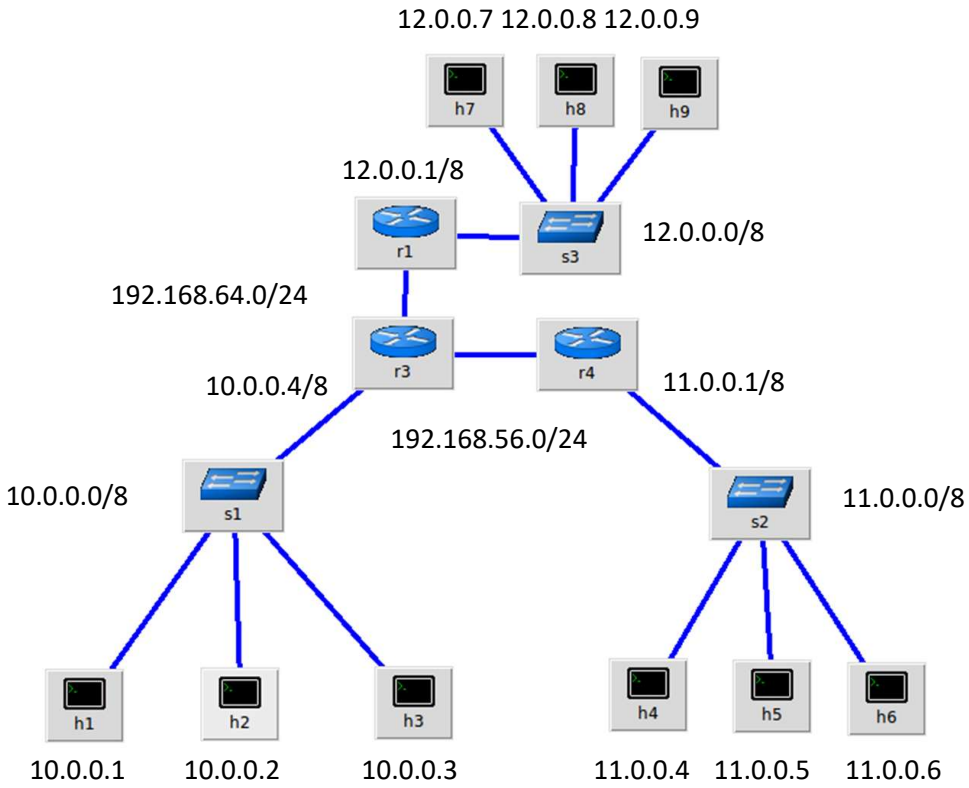
h7 ip route add default via 12.0.0.1

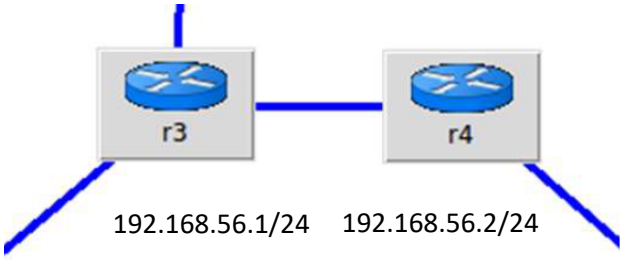
h8 ip route add default via 12.0.0.1

h9 ip route add default via 12.0.0.1

Routingtábla bejegyzések

Feladat 3.6: Mondjuk meg a routereknek, hogy merre találhatók az alhálózatok!





192.168.56.0/24

r3 ip route add 11.0.0.0/8 via 192.168.56.2

r3 ip route add 12.0.0.0/8 via 192.168.64.2

r4 ip route add 10.0.0.0/8 via 192.168.56.1

r4 ip route add 192.168.64.0/24 via 192.168.56.1

r4 ip route add 12.0.0.0/8 via 192.168.56.1

r1 ip route add 192.168.56.0/24 via 192.168.64.1

r1 ip route add 10.0.0.0/8 via 192.168.64.1

r1 ip route add 11.0.0.0/8 via 192.168.64.1

- A közvetlen alhálózati kapcsolatot nem kell megadni (pl.: r3 10-es alhálózatánál)

Beadandó (mininet)

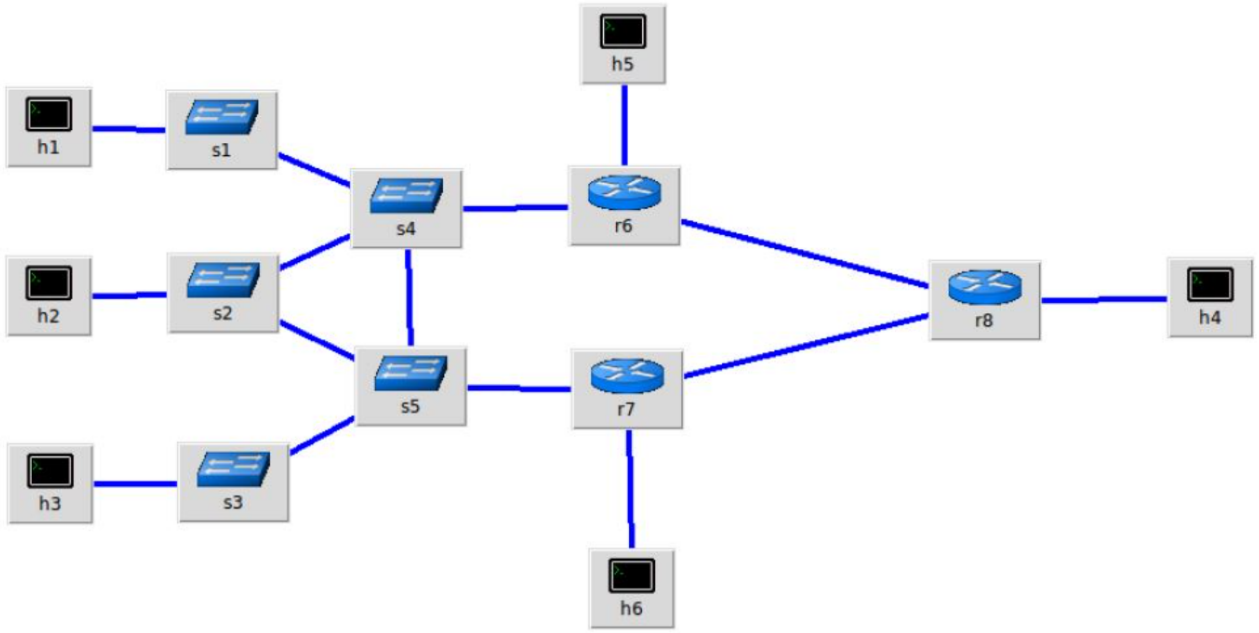
- 2. alfeladat (Default hostok, Routing táblák)

2. alfeladat

Állítsa be a routing táblákat úgy, hogy bármely hosztról bármely másik hoszt elérhető legyen. Mininet terminálban a pingall parancs ad lehetőséget az elérhetőségek gyors tesztelésére!

- A h1 és a h2 állomások az r6 routert, a h3 az r7-es routert használják minden kimenő forgalmuknak a továbbításához!
- Továbbá az r8-as router a 15.0.0.0/8 hálózat eléréséhez a forgalmat az r6-os router felé irányítsa!
- A többi állomás esetén a táblabeállítások egyértelműek, továbbá kérdéses esetben szabadon eldönthető.

Neptun azonosító: Add a házit!



A homework.py fájl tartalmazza az ábrán látható topológia Mininet-es megvalósítását. A feladatokat ezen a fájlon/topológián kell megoldani, annak módosítása nélkül! A topológia vegyesen tartalmaz hosztokat (hX), switcheket (sY) és routereket (rZ) is.

Routing tábla (sw-topo)

- Routing tábla lekérdezése: (pl h1)

```
mininet> h1 route -n
```

- Default route

```
mininet> h1 ip route add default via 10.0.2.254
```

- Routing táblabejegyzés a 10.10.0.0/16

```
mininet> h1 ip route add 10.10.0.0/16 via 10.10.254.254
```

- Bejegyzés

```
mininet> h1 ip route del 10.10.0.0/16
```

Érdekesség:
Host, mint útválasztó eszköz

- A következő példában három hoszt lesz összekötve **h1 – h2 – h3**, és *h2* router-ként lesz konfigurálva
- Ez alapján készült:
  http://csie.nqu.edu.tw/smallko/sdn/mininet-operations.html

Feladat: h2 host
legyen útválasztó
eszköz is!

A `test_router.py` szkript tartalma:

```
#!/usr/bin/python
from mininet.net import Mininet
from mininet.cli import CLI
from mininet.link import Link, TCLink, Intf
from mininet.log import setLogLevel, info
from mininet.node import CPULimitedHost, Host, Node

def myNetwork():
        net = Mininet( topo=None,
                        build=False,
                        ipBase='10.0.0.0/8',
                        link=TCLink)
        info( '*** Adding controller\n' )
        info( '*** Add switches\n')
        info( '*** Add hosts\n')
        h1 = net.addHost('h1', cls=Host, ip='10.0.10.2', defaultRoute=None)
        h2 = net.addHost('h2', cls=Host, ip='10.0.10.1', defaultRoute=None)
        h3 = net.addHost('h3', cls=Host, ip='10.0.20.2', defaultRoute=None)
        info( '*** Add links\n')
        Link(h1, h2)
        Link(h2,h3,intfName1='h2-eth1')
        info( '*** Starting network\n')
        net.build()
        info( '*** Starting controllers\n')
        for controller in net.controllers:
                        controller.start()
        info( '*** Starting switches\n')
        info( '*** Post configure switches and hosts\n')
        CLI(net)
        net.stop()

if name == '__main__':
        setLogLevel( 'info' )
        myNetwork()
```

izsgálódás

- Indítsuk el:

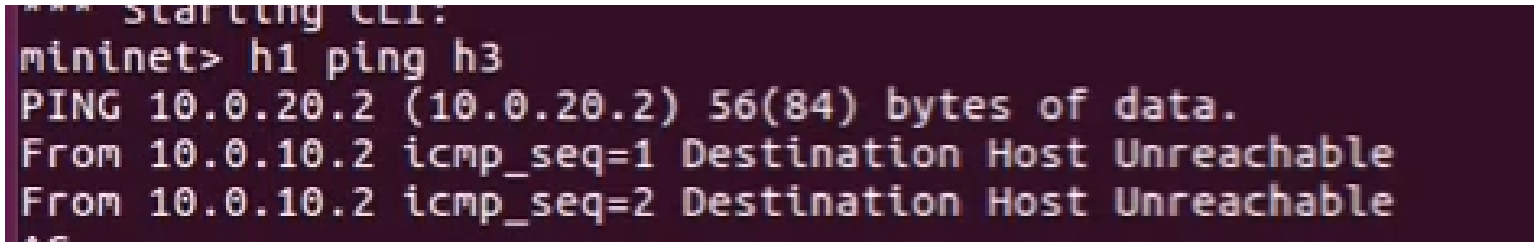
```
root@networks:/home/networks/ComputerNetworks/L2-switching# python test_router.py
mininet>
```

- A h1 h2 h3 hostokon elindíthatunk egy-egy terminált:

```
mininet> xterm h1 h2 h3
```

- Próbáljuk ki, hogy h1 eléri-e h3-at (nem fogja):

```
mininet   h1 ping h3
```



h2 config

- Adjunk IP címet a `h2-eth1` interfésznek:

```
# h2 ip addr add 10.0.20.1/8 dev h2-eth1
```

```
# h2 ifconfig h2-eth1 10.0.20.1 netmask 255.255.255.0
```

```
mininet> h2 ip addr add 10.0.20.1/8 dev h2-eth1
mininet> h2 ifconfig
h2-eth0   Link encap:Ethernet  HWaddr 7e:7a:60:93:83:7c
          inet addr:10.0.10.1  Bcast:10.255.255.255  Mask:255.0.0.0
          inet6 addr: fe80::7c7a:60ff:fe93:837c/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:20 errors:0 dropped:0 overruns:0 frame:0
          TX packets:13 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1244 (1.2 KB)  TX bytes:1046 (1.0 KB)

h2-eth1   Link encap:Ethernet  HWaddr ea:7e:36:c4:fb:61
          inet addr:10.0.20.1  Bcast:0.0.0.0  Mask:255.255.255.0
          inet6 addr: fe80::e87e:36ff:fec4:fb61/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:11 errors:0 dropped:0 overruns:0 frame:0
          TX packets:11 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:866 (866.0 B)  TX bytes:866 (866.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

mininet>
```

h2 config

- Engedélyezzük az IP forwarding-

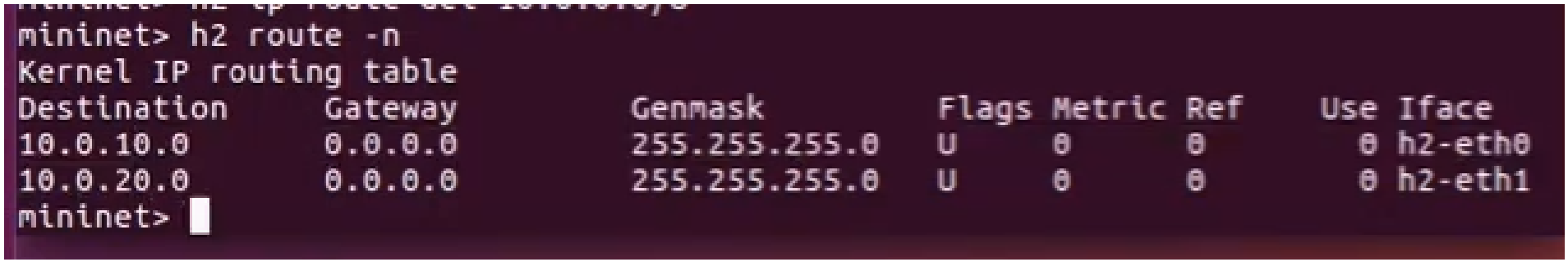
```
mininet> h2 echo 1 > /proc/sys/net/ipv4/ip_forward
```

- Állatsuk be a routing

```
mininet> h2 ip route add 10.0.10.0/24 dev h2-eth0        # h1 <- h3
mininet> h2 ip route add 10.0.20.0/24 dev h2-eth1        # h1 -> h3
```

- Nézzük meg az IP tábláját a h2-nek

```
mininet> h2 route -n
```



Km Să Ko Ő2yfλ∃

- ! Î l·fl·LJŞNJűŜfYŜÎŜűű gűØ2yl·fl·ű l·Řƨdzl Y^Ŝ∃ l· f2l łfλă łűƨłNJ5y^lŜNJŜăÎűÑfΣ l·Y^ŜfèŜű l· Y^Ŝ∃FŜfŜf! λyűŜNJFl·ŐŜπy^fŜKŜű ŜfŞNJyλY

- Km ŘŜFl·dzfű NJ2dzűλy∃Y мл¢л¢мл¢м Kmπ̂ŜűKл

```
Yλyλy̆ŜűƄ Km ip route add default via 10.0.10.1 dev h1-eth0
```

- Töröljük az eredeti route bejegyzést:

```
Yλyλy̆ŜűƄ Km ip route del 10.0.0.0/8
```

- Ko ŘŜFl·dzfű NJ2dzűλy∃Y мл¢л¢нл¢м KoπŜűKл

```
Yλyλy̆ŜűƄ Ko ip route add default via 10.0.20.1 dev h3-eth0
```

```
Yλyλy̆ŜűƄ Ko ip route del 10.0.0.0/8
```

Eredmény

- Ezután nézzük meg az IP routing táblát terminálokban:

```
mininet   h1 route -n
```

```
mininet> h1 route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         10.0.10.1       0.0.0.0         UG    0      0        0 h1-eth0
mininet>
```

- Most már működni fog a ping:

```
mininet> h1 ping h3
```

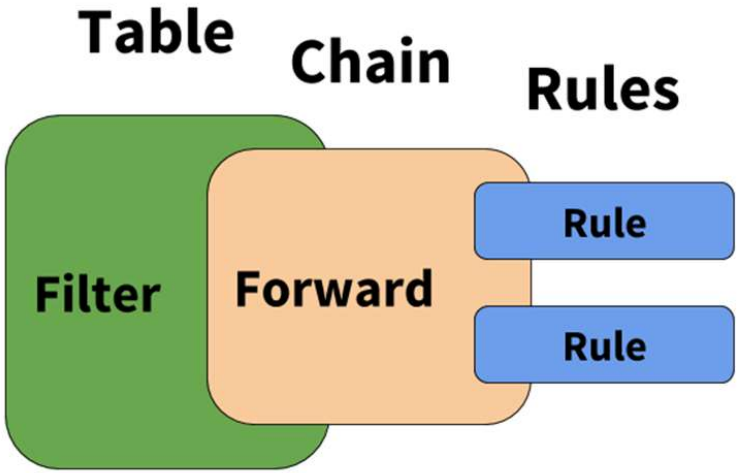
```
mininet> h1 ping h3
PING 10.0.20.2 (10.0.20.2) 56(84) bytes of data.
64 bytes from 10.0.20.2: icmp_seq=1 ttl=63 time=0.076 ms
64 bytes from 10.0.20.2: icmp_seq=2 ttl=63 time=0.080 ms
64 bytes from 10.0.20.2: icmp_seq=3 ttl=63 time=0.061 ms
64 bytes from 10.0.20.2: icmp_seq=4 ttl=63 time=0.063 ms
64 bytes from 10.0.20.2: icmp_seq=5 ttl=63 time=0.080 ms
64 bytes from 10.0.20.2: icmp_seq=6 ttl=63 time=0.065 ms
```

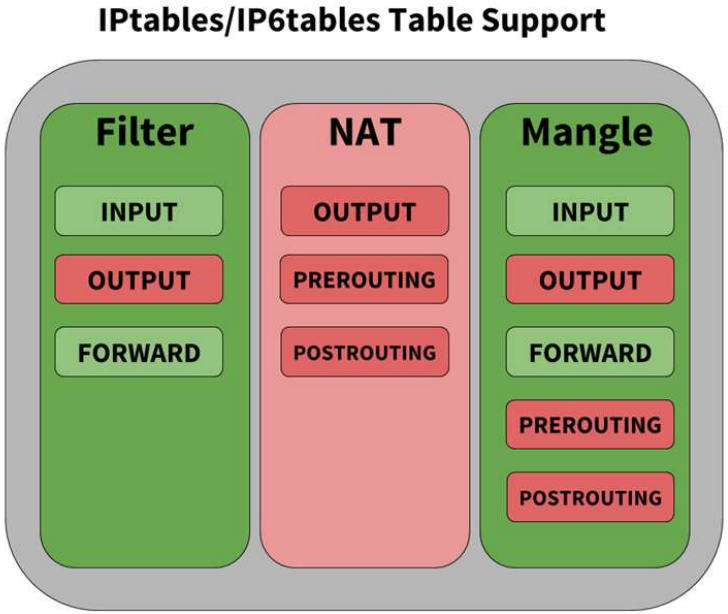
MININET

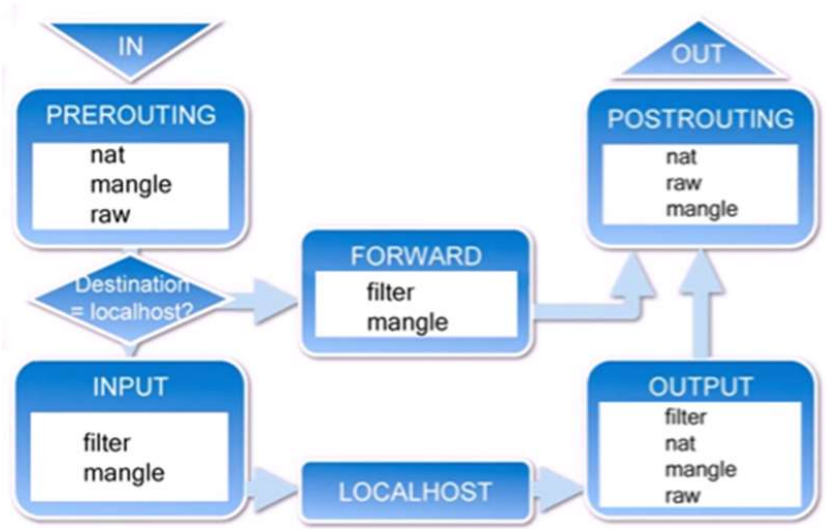
IP táblák (Bead 3. alfeladat)

Iptables

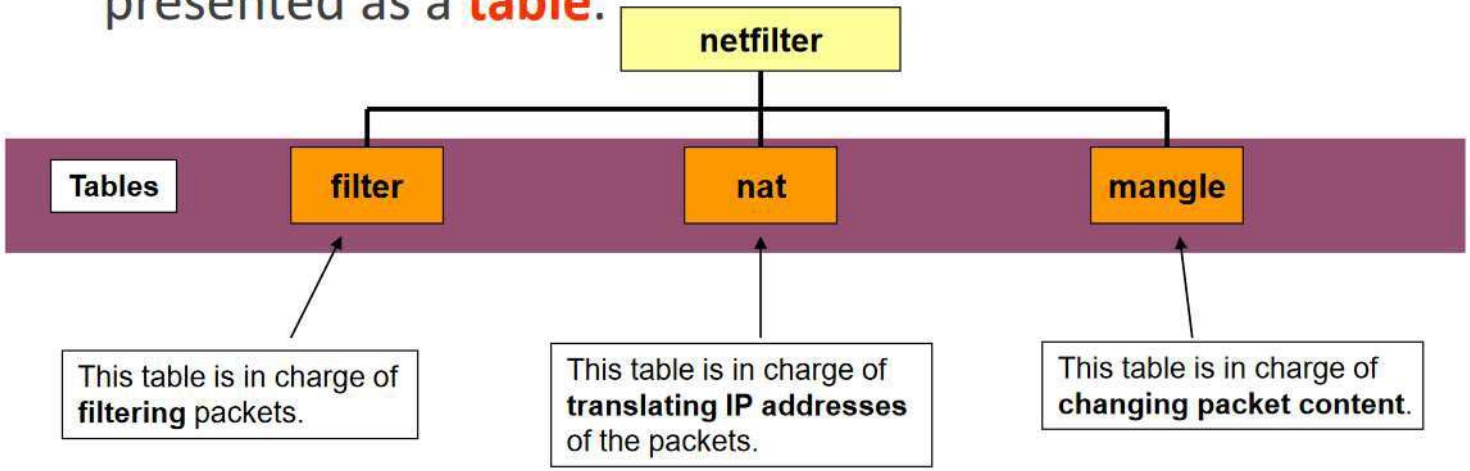
- http://linux-training.be/networking/ch14.html



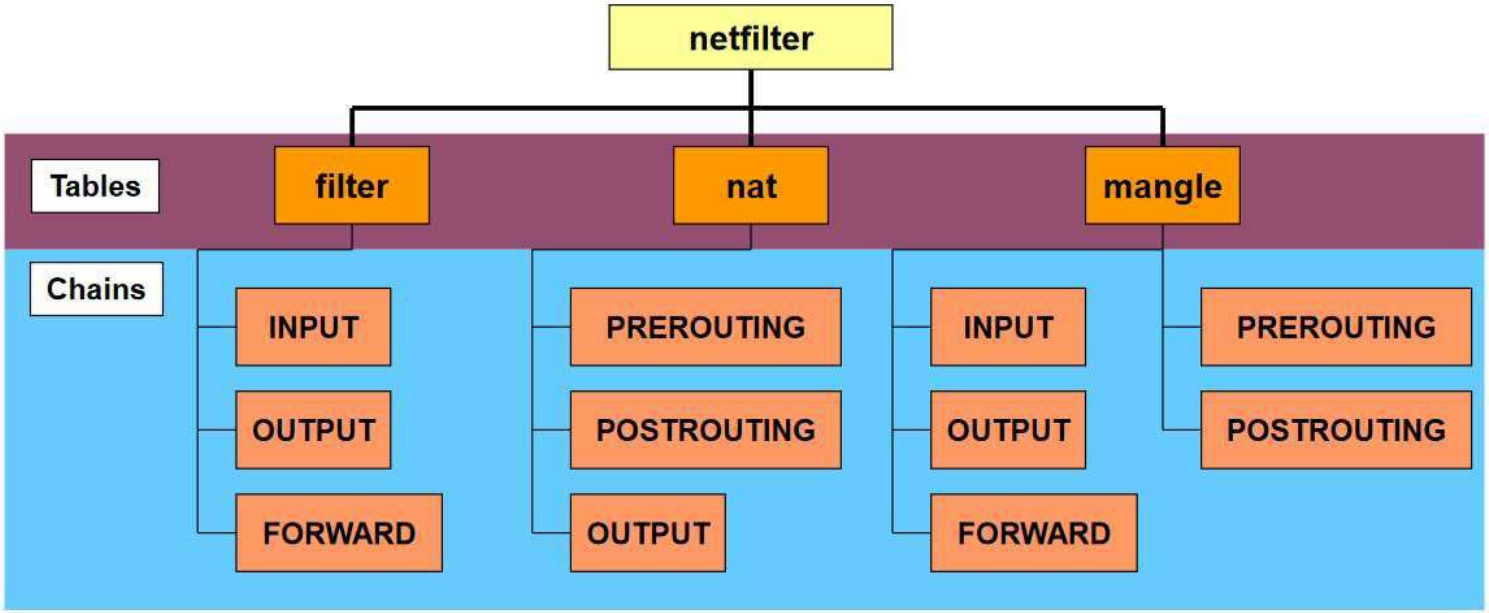


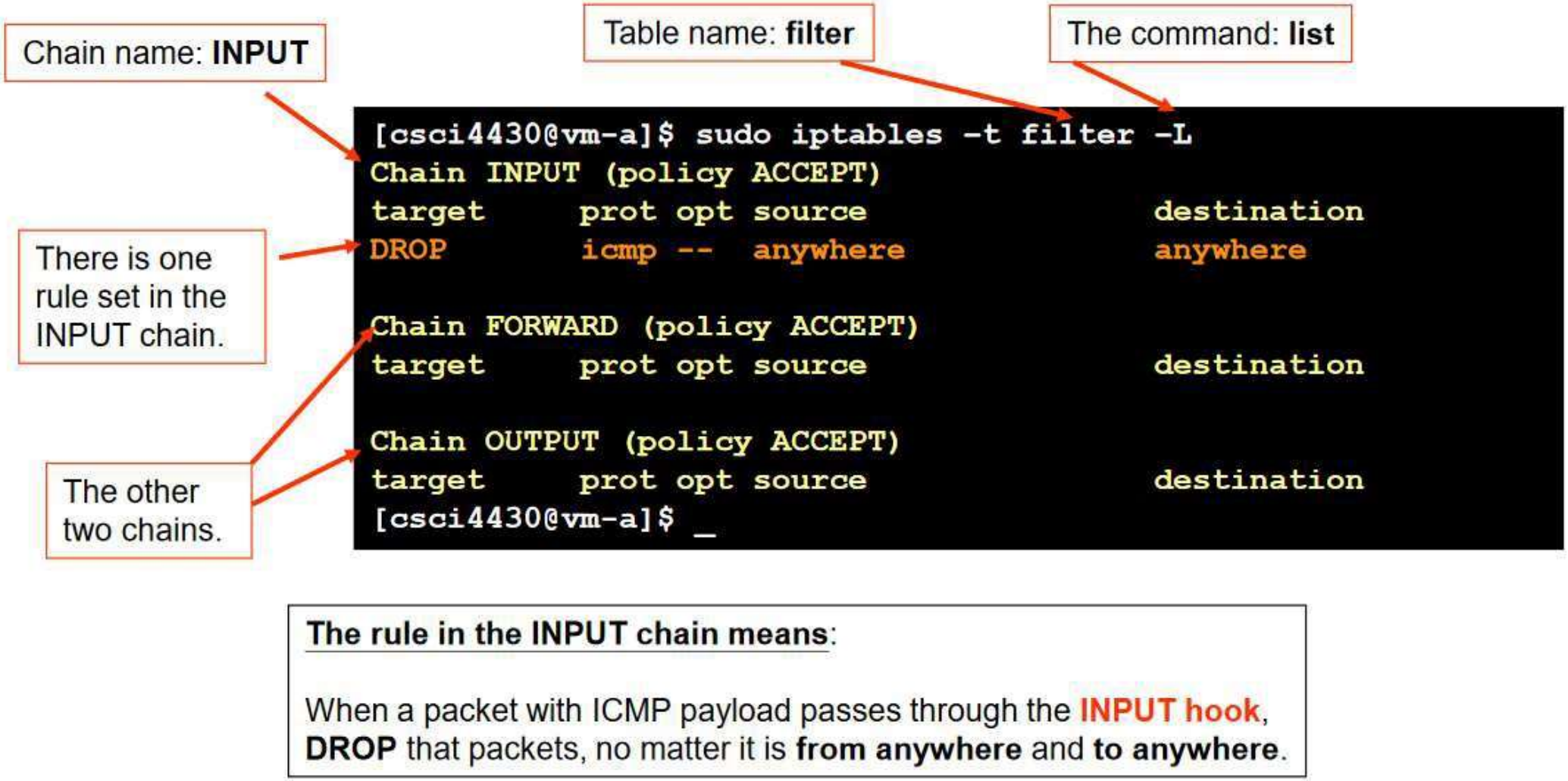


- Each function provided by the netfilter architecture is presented as a **table**.



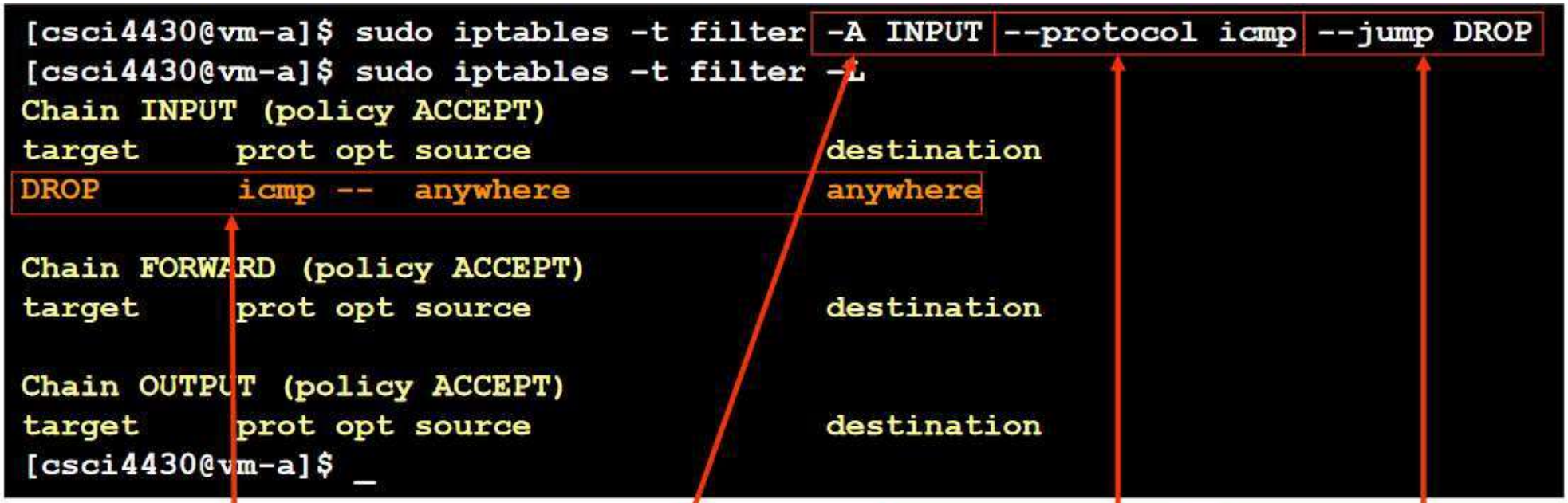
- Under each table, there are a set of **chains**.
  - Under each chain, you can assign a set of **rules**.





The rule in the INPUT chain means:

When a packet with ICMP payload passes through the **INPUT hook**, **DROP** that packets, no matter it is **from anywhere** and **to anywhere**.



This entry shows that a new rule is added to the INPUT chain of the filter table successfully.

Add a new rule to the INPUT chain.

The **protocol** of the packets in which this rule is interested is **ICMP**.

If a packet
(1) passes through the INPUT hook, and
(2) is an ICMP packet,

then the packet **jumps to the target DROP** – **to discard the packet**.

iptables

- Clear all existing rules
  - Flush all entries in the filter table
    - ❖ `iptables -t filter -F`
  - Flush all entries in the nat table
    - ❖ `iptables -t nat -F`
  - Flush all entries in the mangle table
    - ❖ `iptables -t mangle -F`
- List all entries in the nat table
  - ❖ `iptables -t nat -L`
- Always take the manual for reference.

Mininet

- Iptables szabályok mentése:

```
# sudo iptables-save
```

- Ping tiltás szabály felvétele az OUTPUT lánc elejére:

```
# sudo iptables -I OUTPUT -p icmp --icmp-type echo-request -j DROP
```

- Ping tiltás szabály felvétele az OUTPUT lánc végére :

```
# sudo iptables -A OUTPUT -p icmp --icmp-type echo-request -j DROP
```

- Ping tiltás szabály törlése:

```
# sudo iptables -D OUTPUT -p icmp --icmp-type echo-request -j DROP
```

Mininet

- A h1-h2-h3 as feladatunkat egészítsük ki, úgy h1-ről be tudjunk lépni h3-ra és parancsokat kiadni.
- h3 node-on inditsunk el egy ssh deamont

```
mininet> h3 /usr/sbin/sshd
```

- Állítsuk be a h2-es node-on a forwarding szabályt:

```
mininet> h2 iptables  -t nat -A PREROUTING -i h2-eth0 -p tcp -m tcp --dport 2222 -j DNAT --to-destination 10.0.20.2:22
mininet> h2 iptables  -A FORWARD -d 10.0.20.2/32 -p tcp -m tcp --dport 2222 -m state --state NEW,RELATED,ESTABLISHED -j ACCEPT
```

- SSH-zunk be h1-ről a h3-ra a port forwardinggal:

```
mininet> h1 ssh -p 2222 networks@10.0.10.1
```

```
mininet> h3 /usr/sbin/sshd
mininet> h2 iptables -t nat -A PREROUTING -i h2-eth0 -p tcp -m tcp --dport 2222
-j DNAT --to-destination 10.0.20.2:22
mininet> h2 iptables -A FORWARD -d 10.0.20.2/32 -p tcp -m tcp --dport 2222 -m st
ate --state NEW,RELATED,ESTABLISHED -j ACCEPT
mininet> h1 ssh -p 2222 networks@10.0.10.1
networks@10.0.10.1's password: networks

Welcome to Ubuntu 16.04.3 LTS (GNU/Linux 4.10.0-28-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

689 packages can be updated.
522 updates are security updates.

Last login: Tue Nov 23 12:30:38 2021 from 10.0.10.2
networks@networks:~$ ifconfig
h3-eth0   Link encap:Ethernet  HWaddr 16:db:20:20:0d:c2
          inet addr:10.0.20.2  Bcast:10.255.255.255  Mask:255.0.0.0
          inet6 addr: fe80::14db:20ff:fe20:dc2/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:53 errors:0 dropped:0 overruns:0 frame:0
          TX packets:54 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:7095 (7.0 KB)  TX bytes:7017 (7.0 KB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
```

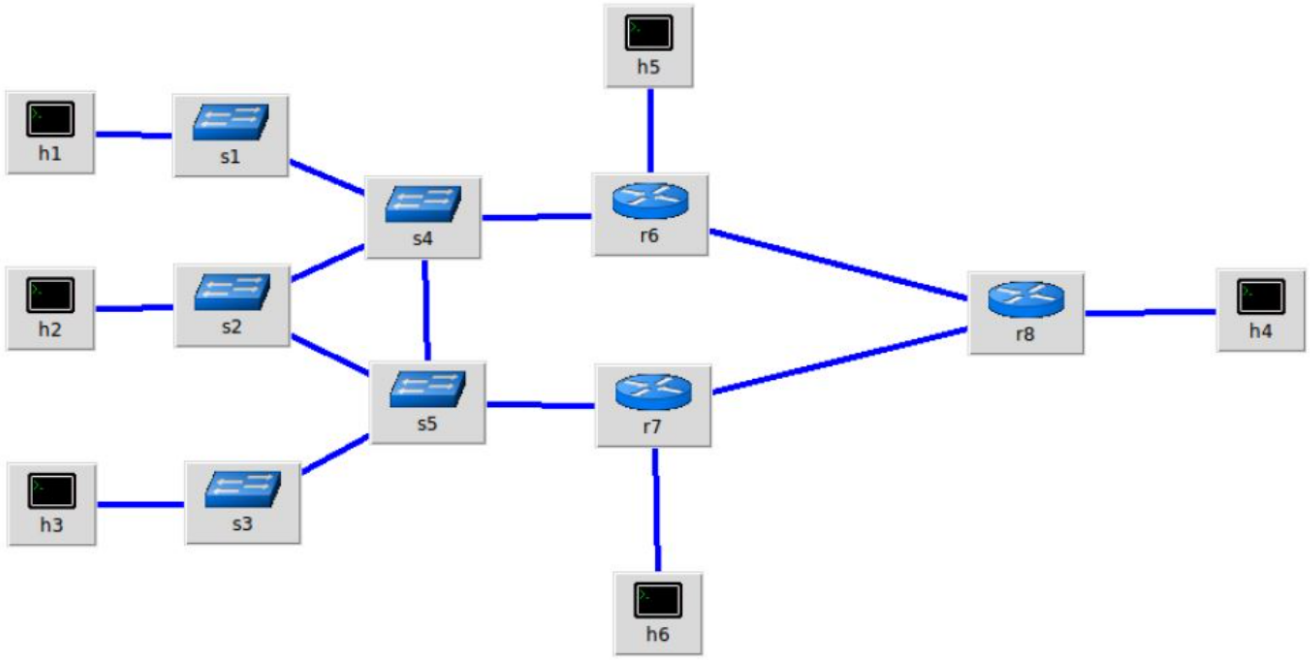
Beadandó (mininet)

- 3. alfeladat (IP táblák)

3. alfeladat

Iptables segítségével állítson be port forwarding (DNAT) szabályt az r6 router h5 hoszt felé eső interfészén úgy, hogy a 122, 222, 322 TCP portokra érkező a forgalmat továbbítsa a h1, h2 h3 22-es TCP portjára. Továbbá a h5 hálózatából jövő és h1-h3 hálózatába menő TCP forgalmakat blokkoljuk (DROP). Adja meg a kiadott iptables parancsokat.

Neptun azonosító: Add a házit!



A homework.py fájl tartalmazza az ábrán látható topológia Mininet-es megvalósítását. A feladatokat ezen a fájlon/topológián kell megoldani, annak módosítása nélkül! A topológia vegyesen tartalmaz hosztokat (hX), switcheket (sY) és routereket (rZ) is.

VÉGE
KÖSZÖNÖM A FIGYELMET!